宇视SDK接口文档

接口概述

方法	说明
初始化相关接口	
startSDK()	SDK启动接口，启动成功后，才可以使用其他API。
setUserInfo()	更新用户相关信息，自动创建云连接。
setClientID()	设置clientID。
自发现搜索相关接口	
startBleSearch()	开启蓝牙搜索。
stopBleSearch()	停止蓝牙搜索。
bleScannPause()	蓝牙扫描暂停。
bleScanResume()	蓝牙扫描恢复。
startWifiSearch()	开启wifi小循环搜索。iOS
wifiScanPause()	WiFi扫描暂停。Android
wifiScanResume()	WiFi扫描恢复。Android
startScanConfigurableDevice()	开始发现可配置的设备。
stopScanConfigurableDevice()	停止发现可配置的设备。
自发现监听相关接口	
addDiscoverDelegate()	添加自发现代理监听。
removeDiscoverDelegate()	移除自发现代理监听。
<DeviceScannerDelegate>	didFindNewDevice() - 发现新增的可配置设备代理方法。
	didRemoveDevice() - 发现有减少的可配置设备代理方法。
	didUpdateDevice() - 发现可配置设备有属性更新代理方法。

获取自发现列表接口	
getDiscoverdDevices()	当前搜索到的可配置的蓝牙设备列表。
绑定相关接口	
bindDeviceByBLE()	BLE配网接口，将设备接入指定的WiFi，并将设备连接到云平台。
bindDeviceBySoftAp()	SoftAp配网接口，将设备接入指定的WiFi，并将设备连接到云平台。
getConfigQRCodeImage()	生成配网二维码。

接口设计

startSDK() - SDK启动接口

代码块

```
/** SDK启动接口 */
- (void)startSDKWithOptions:(StartOptions *)startOptions
                    success:(void(^)(void)) success
                    failure:(void(^)(NSError *error)) failure;
```

setUserInfo() - 设置用户相关信息

代码块

```
/** 设置用户相关信息 */
- (BOOL)setUserInfo:(UserInfo *)userInfo error:(NSError **)error;
```

setClientID() - 设置clientID

代码块

```
/** 设置clientID */
-(void)setClientID:(NSString *)clientID;
```

startBleSearch() - 开启蓝牙搜索

代码块

```
/** 开启蓝牙搜索 */
- (void)startBleSearch:(void(^)(void))success
```

```
3                failure:(void(^)(NSError *error)) failure;
```

stopBleSearch() - 停止蓝牙搜索

代码块

```
1   /** 停止蓝牙搜索 */
2   - (void)stopBleSearch:(void(^)(void))success
3                 failure:(void(^)(NSError *error)) failure;
```

bleScannPause() - 蓝牙扫描暂停

代码块

```
1   /** 蓝牙扫描暂停 */
2   - (void)bleScannPause;
```

bleScanResume() - 蓝牙扫描恢复

代码块

```
1   /** 蓝牙扫描恢复 */
2   - (void)bleScanResume;
```

startWifiSearch() - 开启wifi小循环搜索

代码块

```
1   /** 开启wifi小循环搜索 */
2   - (void)startWifiSearch:(void(^)(void))success
3                   failure:(void(^)(NSError *error)) failure;
```

wifiScanPause() - WiFi扫描暂停

代码块

```
1   /** WiFi扫描暂停 */
2   void wifiScanPause();
```

wifiScanResume() - WiFi扫描恢复

代码块

```
/** WiFi扫描恢复 */
void wifiScanResume();
```

startScanConfigurableDevice() - 开始发现可配置的设备

代码块

```
/** 开始发现可配置的设备 */
- (NSError *)startScanConfigurableDevice;
```

stopScanConfigurableDevice() - 停止发现可配置的设备

代码块

```
/** 停止发现可配置的设备 */
- (NSError *)stopScanConfigurableDevice;
```

addDiscoverDelegate() - 停止发现可配置的设备

代码块

```
/** 添加自发现代理监听 */
- (void)addDiscoverDelegate:(id<DeviceScannerDelegate>)delegate;
```

removeDiscoverDelegate() - 移除自发现代理监听

代码块

```
/** 移除自发现代理监听 */
- (void)removeDelegate:(id<DeviceScannerDelegate>)delegate;
```

<DeviceScannerDelegate> - 自发现代理监听

代码块

```
/** 代理，当扫描到的可配置设备列表发生变化或设备有属性更新时触发 */
@protocol DeviceScannerDelegate <NSObject>

/** 发现新增的可配置设备 */
- (void)didFindNewDevice:(DiscoverDevice *)device;

/** 发现有减少的可配置设备 */
```

```
- (void)didRemoveDevice:(DiscoverDevice *)device;

/** 发现可配置设备有属性更新 */
- (void)didUpdateDevice:(DiscoverDevice *)device;
@end
```

getDiscoverdDevices() - 当前搜索到的可配置的蓝牙设备列表

代码块

```
/** 当前搜索到的可配置的蓝牙设备列表 */
-(NSArray<DiscoverDevice*> *)getDiscoverdDevices;
```

bindDeviceByBLE() - BLE配网接口

代码块

```
/** BLE配网接口 */
+ (void)bindDeviceByBLE:(DiscoverDevice *)device
                bindInfo:(BLEBindInfo *)bindInfo
          progressNotify:(void(^)(BindProgressInfo *bindProgressInfo))progressNotify
                 success:(void(^)(BindDevice *device))success
                 failure:(void(^)(NSError *error))failure;
```

bindDeviceBySoftAp() - SoftAp配网接口

代码块

```
/** SoftAp配网接口 */
+ (void)bindDeviceBySoftAp:(SoftApBindInfo *)softApBindInfo
             progressNotify:(void(^)(BindProgressInfo *bindProgressInfo))progressNotify
       switchNetworkNotify:(void(^)(void))switchNetworkNotify
                   success:(void(^)(BindDevice *device))success
                   failure:(void(^)(NSError *error))failure;
```

getConfigQRCodeImage() - 获取配网二维码

代码块

```
/** 获取配网二维码 */
+ (void)getConfigQRCodeImage:(ScanQRCodeBindInfo *)bindInfo
```

```
                connectToCloudNotify:(void(^)(void))connectToCloudNotify
                             success:(void(^)(UIImage *QRImage))success
                             failure:(void(^)(NSError *error))failure;
```

数据模型

StartOptions - SDK的启动参数

代码块

```
/** uSDK的启动参数 */
@interface StartOptions : NSObject

/** 开发者网站上申请获得的appID，必填 */
@property (nonatomic, copy) NSString *appID;

/** 开发者网站上申请获得的appKey，必填 */
@property (nonatomic, copy) NSString *appKey;

/** 开发者网站上申请获得的secretKey，必填 */
@property (nonatomic, copy) NSString *secretKey;

/** 默认为YES，表示开启BLE可控设备搜索，如果不想开启，则将该属性置为NO */
@property (nonatomic, assign) BOOL enableBLEControllableSearch;

/** 默认为YES，表示开启wifi小循环设备搜索，如果不想开启，则将该属性置为NO */
@property (nonatomic, assign) BOOL enableWifiSearch;

@end
```

UserInfo - 用户信息类

代码块

```
/** 用户信息 */
@interface UserInfo : NSObject

/** App登录后获取到的userId，必填 */
@property (nonatomic, copy) NSString *userID;

/** App登录后获取到的token，必填 */
@property (nonatomic, copy) NSString *userToken;

@end
```

DiscoverDevice - 自发现设备信息类

代码块

```objc
/** 自发现设备信息类 */

@interface uSDKDeviceInfo : NSObject

/** 设备ID */
@property (atomic, copy ,readonly) NSString* deviceID;

/** 设备mac */
@property (nonatomic, copy ,readonly) NSString* mac;

/** 产品编码 */
@property (nonatomic, copy, readonly) NSString *productCode;

/** 设备的类型唯一识别码 */
@property (nonatomic, copy ,readonly) NSString* typeID;

/** 应用分类编码 */
@property (nonatomic, copy ,readonly) NSString* appTypeCode;

/** 应用类型名称  需要跟海尔设备保持一致*/
@property (nonatomic, copy, readonly) NSString* apptypeName;

/** 设备热点名称 */
@property (nonatomic, copy ,readonly) NSString* hotSpotName;

/** 设备热点前缀+mac后四位 */
@property (nonatomic, copy, readonly) NSString *name;

/** 设备大类分类 需要跟海尔设备保持一致*/
@property (nonatomic, assign ) uSDKDeviceTypeConst bigType;

/** 设备中类 需要跟海尔设备保持一致*/
@property (nonatomic, assign ,readonly) NSInteger middleType;

/** 设备支持的配置方式，用于自发现设备 */
@property (nonatomic, assign, readonly) DeviceConfigType configType;

/** 设备当前的可配置状态 */
@property (nonatomic, assign, readonly) DeviceConfigState configState;

/** 蓝牙广播包中的RSSI，用于判断信号强度 负值，越接近0信号越强 */
@property (nonatomic, assign) NSInteger RSSI;

```

```
/** 信号强度等级 取值范围 1 ~ 5,    5 优，4 良，3 中，2 差，1 极差    默认值0代表 无效 */
@property (nonatomic, assign, readonly) NSInteger BLESignalLevel;

/** 设备支持配网 WiFi频段 */
@property (nonatomic, assign, readonly) NSString* wifiFreqBrand;

/** 软件版本短id */
@property (nonatomic, copy, readonly) NSString * swShortId;

--- 如果不能走海尔serve请求设备信息，需上报 设备图标 设备昵称 ---
/** 设备url图标  */
@property (nonatomic, copy ,readonly) NSString* deviceIcon

/** 设备昵称，用于搜索发现阶段展示 */
@property (nonatomic, copy ,readonly) NSString* nickname;

/** 设备是否支持 WIFI&BLE 批量绑定 */
@property (nonatomic, assign, readonly) BOOL batchBindSupport;

/** 快连设备信号强度等级 取值范围 1 ~ 5,    5 优，4 良，3 中，2 差，1 极差    默认值0代表 无效 */
@property (nonatomic, assign, readonly) NSInteger QCBLESignalLevel;

/** 设备的可控状态 */
@property (nonatomic, assign, readonly) DeviceControlState controlState;

//** 标识快联设备是否需要确权 */
@property (nonatomic, assign, readonly) BOOL isNeedAuth;

/** 是否有工单 */
@property (nonatomic, assign, readonly) BOOL workOrder;

/** 设备上次绑定状态 */
@property (nonatomic, assign, readonly) NSInteger lastBindStatus;

@end
```

DeviceConfigType - 设备支持的配置方式

代码块

```
/** 设备支持的配置方式 */
typedef NS_OPTIONS(NSUInteger, DeviceConfigType) {

    /** BLE配置方式 */
```

```
    BLE,

    /** softAp配置方式 */
    SoftAP
};
```

DeviceConfigState - 设备的可配置状态

代码块

```
/** 设备当前可配置状态 */
typedef NS_ENUM(NSUInteger, DeviceConfigState) {

    /** 设备当前状态已配置 */
    Configured,
    /** 设备当前状态可配置 */
    Configurable,
    /** 设备当前状态可触发进配置 */
    TriggerConfigurable,
    /** 设备当前状态已绑定可配网 */
    UpRouterable
};
```

DeviceControlState - 设备的可控状态

代码块

```
/** 设备的可控状态 */
typedef NS_ENUM(NSUInteger, DeviceControlState) {

    /** 无可控状态 */
    None,
    /** 可控制 */
    Controllable,
    /** 靠近可控制 */
    NearControllable,
    /** 可控制且设备侧鉴权通过 */
    ControllableAndAuthorized
};
```

BLEBindInfo - BLE设备绑定参数类

代码块

```objc
/** BLE设备绑定参数类 */
@interface BLEBindInfo : NSObject

/** 接入点WIFI名称，必填 */
@property (nonatomic, copy) NSString *ssid;

/** 接入点WIFI的bssid */
@property (nonatomic, copy) NSString *bssid;

/** 接入点WIFI密码，支持无密码的情况，但如果WIFI有密码，则密码长度必须在[8,64]之间 */
@property (nonatomic, copy) NSString *password;

/** 超时时间（单位是秒，范围为30秒-180秒），默认90秒 */
@property (nonatomic, assign) NSTimeInterval timeoutInterval;

@end
```

BindProgressInfo - 配置绑定接口进度信息

代码块

```objc
/** 进度信息 */
@interface BindProgressInfo : NSObject

@property (nonatomic, copy) NSString *deviceID;
@property (nonatomic, copy) NSString *typeID;
@property (nonatomic, assign) BindProgressState bindProgress;

@end
```

BindProgressState - 进度枚举

代码块

```objc
/** 设备配网进度 */
typedef NS_ENUM(NSUInteger, BindProgressState) {

    /** 开始连接设备 */
    ConnectDevice,

    /** 开始发送配置信息 */
    SendConfigInfo,

    /** 开始绑定设备 */
    BindDevice,
```

```
12
13      /** 绑定设备成功 */
14      BindDeviceSuccess,
15
16      /** 绑定设备失败 */
17      BindDeviceFailure,
18  };
```

BindDevice - 绑定成功设备信息

代码块

```
1   /** uSDK设备实体类，用于描述设备的基本信息 */
2   @interface BindDevice : NSObject
3
4   /** 设备ID */
5   @property (nonatomic, copy ,readonly) NSString* deviceID;
6
7   /** 设备绑定名称 */
8   @property (nonatomic, copy, readonly) NSString *deviceName;
9
10  /** 设备的类型唯一识别码 */
11  @property (nonatomic, copy ,readonly) NSString* typeID;
12
13  @end
```

SoftApBindInfo - softAp配置绑定信息类

代码块

```
1   /** softAp配置绑定信息类 */
2   @interface SoftApBindInfo : NSObject
3
4   /** 要配置到的SSID名称，必填，SSID最长支持32个字符 */
5   @property (nonatomic, copy) NSString* ssid;
6
7   /** 要配置的路由器的MAC */
8   @property (nonatomic, copy) NSString* bssid;
9
10  /** 要配置的SSID对应的密码，必填，可以为空密码，如果有密码，则长度必须>=8 && <=64 */
11  @property (nonatomic, copy) NSString* password;
12
13  /** 设备SoftAp热点的SSID，必填 */
14  @property (nonatomic, copy) NSString* iotDeviceSSID;
15
```

```objc
/** 配置超时时间（单位是秒，范围为30秒-120秒），默认60秒 */
@property (nonatomic, assign) NSTimeInterval timeoutInterval;

/** 配置方式为非安全或安全，默认为非安全 */
@property (nonatomic, assign) BOOL security;

@end
```

ScanQRCodeBindInfo - 摄像头扫码绑定信息类

代码块

```objc
/** 摄像头扫码绑定信息类 */
@interface ScanQRCodeBindInfo : NSObject

/** 成品编码，必填 */
@property (nonatomic, copy) NSString *productCode;

/** 要配置到的SSID名称，必填，SSID最长支持32个字符 */
@property (nonatomic, copy) NSString *ssid;

/** 要配置的SSID对应的密码，必填，可以为空密码，如果有密码，则长度必须>=8 && <=64 */
@property (nonatomic, copy) NSString *password;

/** 超时时间 */
@property (nonatomic,assign) NSTimeInterval timeoutInterval;

/** 设备的typeID 必填 */
@property (nonatomic, copy) NSString *typeID;

@end
```